Stroke Prediction using Random Forest

Minor Project Report — Udeep Karmakar

1. Objective and Dataset

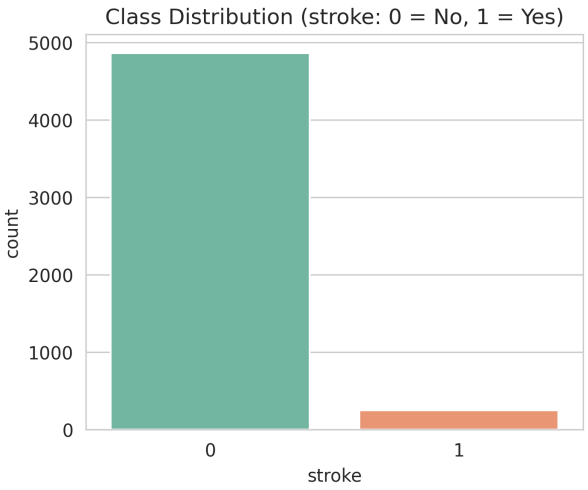
Workflow overview: (1) exploratory data analysis, (2) preprocessing (null handling, encoding, 80:20 split), (3) Random Forest development with hyperparameter justification via grid search, (4) evaluation with the five required metrics, and (5) conclusion. Each step is explained in the sections below.

The objective is to build a Random Forest classifier that predicts whether a patient is likely to suffer a stroke from demographic and health attributes, and to evaluate it with metrics suited to a heavily imbalanced medical problem.

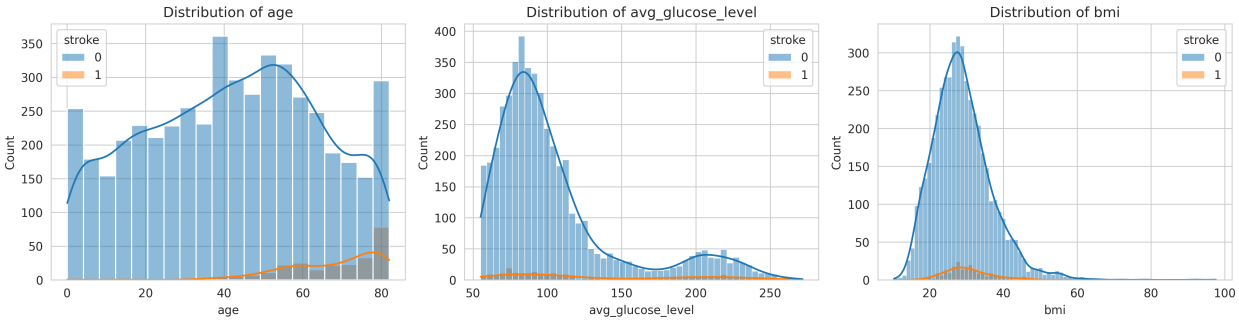
The dataset is the Kaggle *Stroke Prediction Dataset*: 5,110 records and 12 columns (id, gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status, stroke). The target is **stroke** (1 = stroke, 0 = no stroke). Only 249 records (4.87%) are stroke cases, so the data is strongly imbalanced, and 201 values of **bmi** are missing.

2. Exploratory Data Analysis

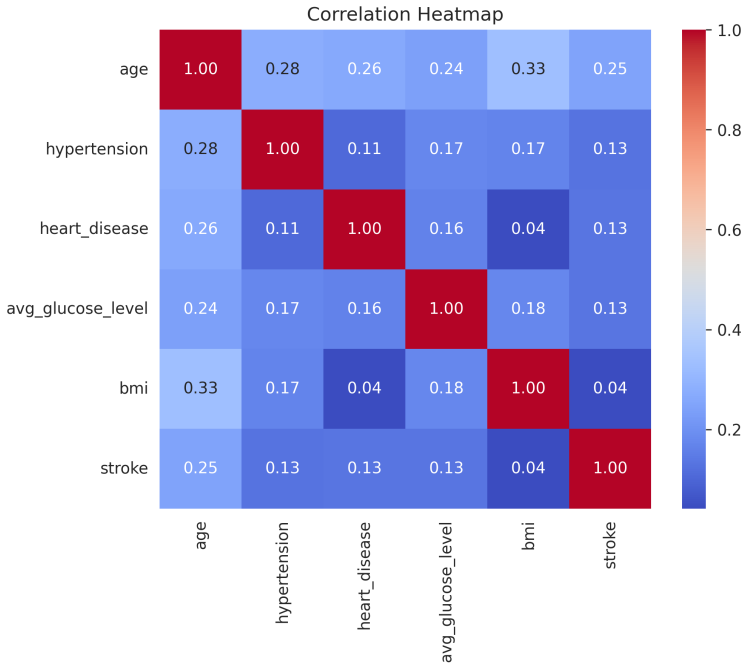
Class balance: 95.13% of patients have no stroke versus 4.87% with stroke. A model predicting 'no stroke' for everyone would reach about 95% accuracy while detecting no strokes, so accuracy alone is misleading and F1 and recall are used instead.



Distributions: stroke cases are concentrated at older ages, and higher average glucose levels are more common among them. BMI is right-skewed.



Correlation: among numeric features, **age** has the strongest correlation with stroke (r = 0.245). Hypertension, heart disease and glucose level show weaker positive correlations.



3. Data Cleaning and Preprocessing

Missing values: the 201 missing **bmi** values were filled with the median (28.1). The median was chosen over the mean (28.9) because BMI is right-skewed and the median is robust to outliers.

Dropped column: id is only an identifier and has no predictive value.

Encoding: categorical features (gender, ever_married, work_type, Residence_type, smoking_status) were one-hot encoded with drop_first=True to avoid redundant columns, giving 16 features.

Split: 80:20 stratified train/test split (4,088 training and 1,022 test rows, random_state=42), keeping the ~5% stroke ratio in both sets.

Scaling: no scaling was applied; Random Forest splits on thresholds rather than distances, so scaling does not change its results.

Imbalance: handled with `class_weight='balanced'`, which up-weights the minority class during training without altering the test data.

4. Model and Hyperparameter Justification

A grid search with 3-fold cross-validation on the training set tuned **n_estimators** (100, 200, 300) and **max_depth** (3, 5, 8, 12, None), scored on **F1** because of the imbalance. The best combination was **n_estimators = 100, max_depth = 5** (cross-validated F1 = 0.2252).

n_estimators	max_depth	CV F1
100	5	0.2252
200	5	0.2219
300	5	0.2201

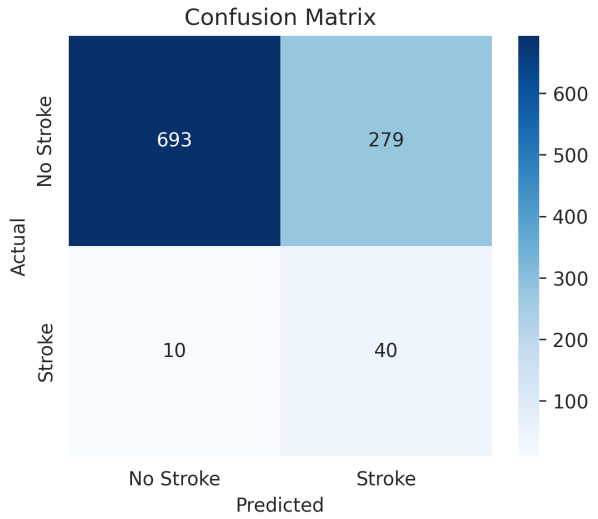
100	8	0.2056
200	8	0.2003

Shallow trees performed best. Deeper trees (and unlimited depth) memorise the few stroke cases and overfit, while depth 5 forces simpler, more general splits. Adding more trees beyond 100 gave no gain, so 100 trees is sufficient and cheaper to train.

5. Results

Metric	Score
Accuracy	0.7172
Precision	0.1254
Recall	0.8000
F1-Score	0.2168
ROC-AUC	0.8419

5.1 Confusion Matrix



On the 1,022 test patients: 693 true negatives, 279 false positives, 10 false negatives and 40 true positives. The model caught 40 of the 50 actual stroke cases.

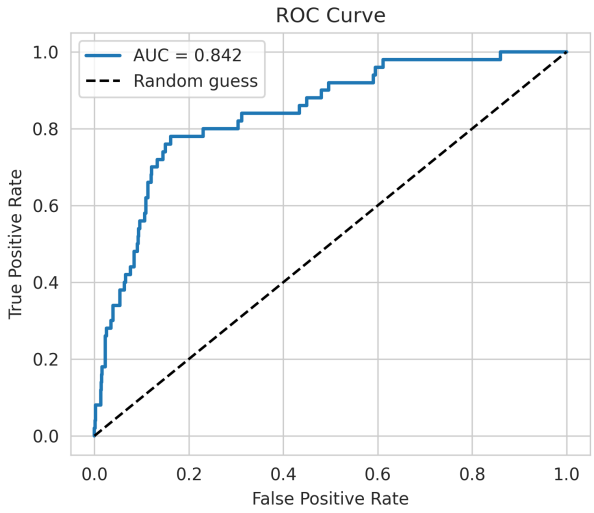
5.2 Precision and Recall

Recall (0.80) is high: the model finds 80% of real stroke cases, which is the priority in a medical screening setting where a missed stroke (false negative) is far costlier than a false alarm. **Precision (0.13)** is low: only about 1 in 8 patients flagged as stroke actually has one, because 279 healthy patients were flagged. This is the expected trade-off of class balancing on data with ~5% positives; it is acceptable for a first-stage screening tool but not for a final diagnosis.

5.3 F1-Score

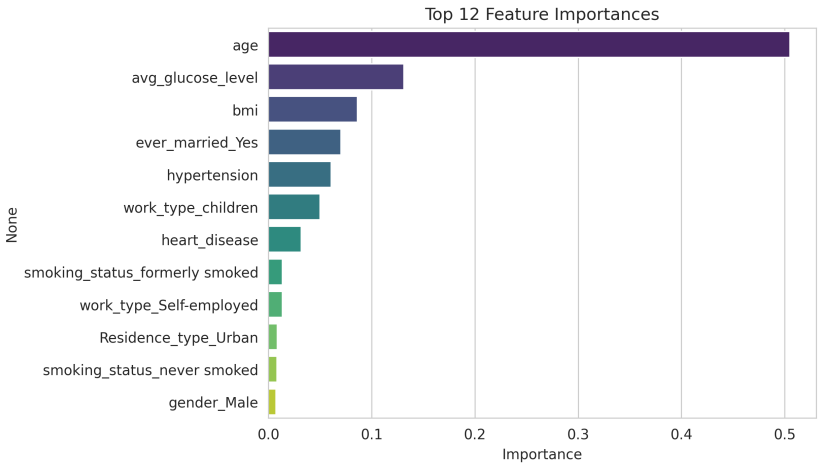
The F1-score of 0.2168 is the harmonic mean of the low precision and high recall. It is modest in absolute terms, which honestly reflects how hard the problem is, but it is a far more informative measure than the 71.7% accuracy, which is itself lower than the 95% a trivial majority-class model would score.

5.4 ROC-AUC



ROC-AUC of 0.842 shows the model ranks stroke patients above non-stroke patients about 84% of the time, well above the 0.5 of random guessing. The ranking ability is good even though the default 0.5 threshold gives low precision.

5.5 Feature Importance



Age dominates (importance 0.51), followed by **avg_glucose_level** (0.13), **bmi** (0.09), **ever_married** (0.07) and **hypertension** (0.06). This matches the EDA, where age had the strongest correlation. The importance of *ever_married* and *work_type_children* is most likely a proxy for age rather than a causal effect.

6. Conclusion

Strengths: the model achieves high recall (0.80) and good ranking performance (AUC 0.84), identifies clinically sensible risk factors (age, glucose, hypertension), and uses a leakage-free pipeline (stratified split, tuning only on training data).

Weaknesses: precision is very low (0.13) and F1 only 0.22, so the model generates many false alarms. With only 249 positive cases (50 in the test set), results are sensitive to the particular split. Median imputation ignores relationships between features, and the grid search was limited to two hyperparameters.

Possible improvements: tune the decision threshold using the precision-recall curve; try SMOTE or under-sampling; use k-fold cross-validation for the final evaluation; impute BMI with a model (e.g. KNN or iterative imputation); tune more hyperparameters (min_samples_leaf, max_features); and compare with gradient boosting such as XGBoost or LightGBM.